

Implementacja Modelu Perceptronu Wielowarstwowego

Natalia Choszczyk
nr indeksu: 327267

Spis treści

1	Wprowadzenie	2
2	NN1: bazowa implementacja	2
2.1	Opis zadania	2
2.2	Implementacja	2
2.3	Wyniki i wnioski	2
3	NN2: implementacja propagacji wstecznej błędu	4
3.1	Opis zadania	4
3.2	Implementacja	4
3.3	Wyniki i wnioski	4
4	NN3: implementacja momentu i normalizacji gradientu	6
4.1	Opis zadania	6
4.2	Implementacja	6
4.3	Wyniki i wnioski	6
5	NN4: rozwiązywanie zadania klasyfikacji	9
5.1	Opis zadania	9
5.2	Implementacja	9
5.3	Wyniki i wnioski	9
6	NN5: testowanie różnych funkcji aktywacji	11
6.1	Opis zadania	11
6.2	Implementacja	11
6.3	Wyniki i wnioski	11
7	NN6: zjawisko przeuczenia i regularyzacja	14
7.1	Opis zadania	14
7.2	Implementacja	14
7.3	Wyniki i wnioski	14

1 Wprowadzenie

Celem laboratorium było zrozumienie budowy i działania perceptronu wielowarstwowego (MLP) oraz analiza wpływu hiperparametrów na proces uczenia. Zadanie polegało na zaimplementowaniu algorytmu sieci neuronowych od podstaw, bez wykorzystania gotowych bibliotek i stopniowym dodawaniu kolejnych funkcjonalności i możliwości. Wykonując kolejne części, ważna była nie tylko implementacja, ale też odpowiednie wytrenowanie sieci na różnych przykładach z bardzo wysoką dokładnością.

2 NN1: bazowa implementacja

2.1 Opis zadania

Zadanie polegało na zaimplementowaniu sieci MLP z możliwością konfiguracji liczby warstw, neuronów oraz wag, w tym biasów. Sieć wykorzystywała funkcję aktywacji sigmoidalną w warstwach ukrytych oraz funkcję liniową na wyjściu. Implementacja miała umożliwiać łatwą modyfikację architektury oraz funkcji aktywacji.

2.2 Implementacja

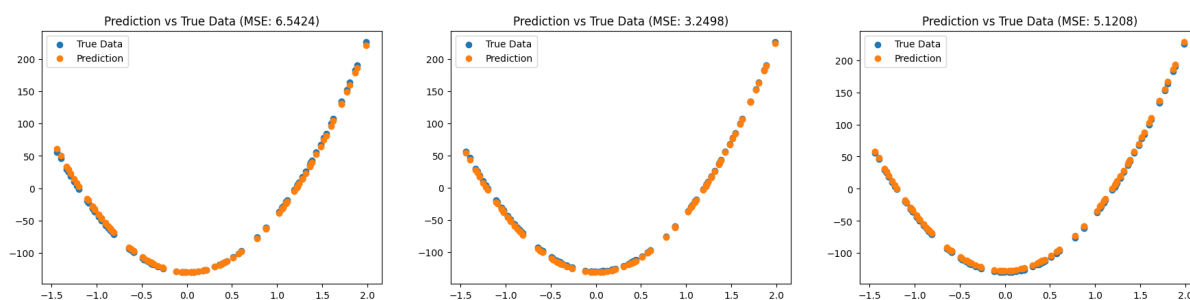
Bazowa implementacja opiera się na stworzeniu klasy `MSE`, która zawiera metody:

- `forward` - odpowiada za przejście danych w przód przez sieć
- `predict` - zwraca wynik przejścia `forward` na ostatniej warstwie
- `set_weights_and_biases` - pozwala na ręczne ustawienie wartości wag i bias
- `mse` - zwraca wartość MSE

2.3 Wyniki i wnioski

Eksperymenty zostały przeprowadzone na dwóch zbiorach: *square-simple* oraz *steps-large*. Dodatkowo zostały przetestowane trzy różne architektury: jedna warstwa ukryta - 5 neuronów, jedna warstwa ukryta - 10 neuronów, dwie warstwy ukryte, po 5 neuronów każda. Celem było uzyskanie wartości błędu średniokwadratowego (MSE) poniżej 9 na obu zbiorach. Było to bardzo duże wyzwanie wymagające wielu testów i kombinacji. Stosowane wagi były dobierane ręcznie, co stanowiło największe wyzwanie. Poniżej przedstawię wizualizację uzyskanych wyników oraz wykorzystane parametry.

Wyniki dla zbioru *square-simple*

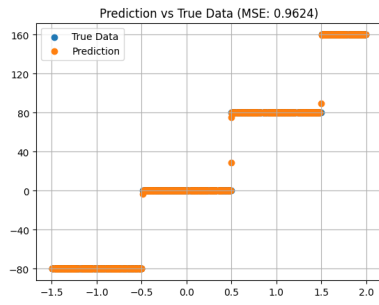


Rysunek 1: Architektura 1-5-1

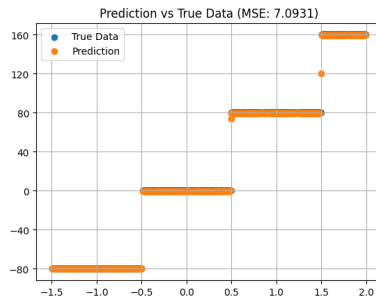
Rysunek 2: Architektura 1-10-1

Rysunek 3: Architektura 1-5-5-1

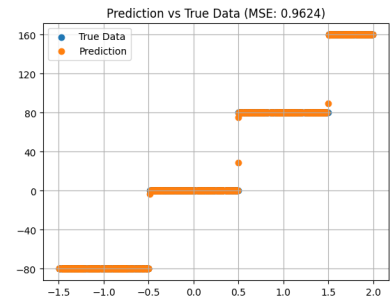
Wyniki dla zbioru *steps-large*



Rysunek 4: Architektura 1-5-1



Rysunek 5: Architektura 1-10-1



Rysunek 6: Architektura 1-5-5-1

zbiór danych	architektura	MSE
<i>square-simple</i>	1-5-1	6.5424
<i>square-simple</i>	1-10-1	3.2498
<i>square-simple</i>	1-5-5-1	5.1208
<i>steps-large</i>	1-5-1	0.9624
<i>steps-large</i>	1-10-1	7.0931
<i>steps-large</i>	1-5-5-1	0.9624

Tabela 1: Uzyskane wyniki MSE

3 NN2: implementacja propagacji wstecznej błędu

3.1 Opis zadania

Celem zadania była implementacja algorytmu propagacji wstecznej błędu (backpropagation). Został on zaimplementowany zgodnie ze wzorami przedstawionymi na wykładzie oraz laboratoriach.

Sieć miała umożliwiać uczenie na podstawie dostarczonych danych treningowych oraz wizualizację wartości wag w kolejnych iteracjach procesu uczenia. Dla każdej warstwy zostaje wyświetlony wykres wartości wag w poszczególnych epokach.

Sieć można trenować na dwa sposoby. Z wykorzystaniem batch lub nie. Oznacza to, że możemy aktualizować wartości wag po przejściu określonej części zbioru treningowego przez sieć - wiele razy w trakcie jednej epoki lub dopiero po przejściu całego zbioru przez sieć - raz dla każdej epoki.

Wagi mogą być inicjalizowane na trzy sposoby: z rozkładu jednostajnego na przedziale $[0, 1]$, za pomocą metody He lub Xavier.

3.2 Implementacja

Kod potrzebny do zainicjalizowania oraz wytrenowania sieci składa się z trzech plików: `MLP.py`, `Layers.py` oraz `ActivationFunctions`.

Plik `MLP.py` zawiera definicję klasy `MLP`, a w niej metody `feedforward`, `backpropagate`, `train`, `predict`, `set_weights_and_biases`, `mse` oraz metody do generowania wykresów: `plot_loss`, `plot_weights`, `plot_predictions`

Plik `Layers.py` zawiera definicję klasy `Layer`, która dotyczy poszczególnych warstw sieci, inicjalizuje wagi oraz posiada metodę `forward`.

Plik `ActivationFunctions` zawiera definicje funkcji aktywacji oraz ich pochodnych.

Aby stworzyć sieć, a potem ją wytrenować, musimy ją zainicjalizować, gdzie definiujemy liczbę neuronów w każdej warstwie oraz metodę losowania wag początkowych. Jako metodę losowania wag początkowych możemy wybrać: `uniform`, `xavier` lub `he`.

Następnie na stworzonym obiekcie klasy `MLP` wywołujemy metodę `train`, w której określamy liczbę epok, learning rate oraz rozmiar batcha. Metoda ta przechodzi przez wszystkie epoki oraz przez wszystkie próbki batch, wykonuje backpropagację oraz aktualizuje wartości wag i biasów po każdym przejściu batcha. Następnie tworzy predykcje i oblicza błąd MSE dla każdej epoki. Rysuje również wykres wag i wykres MSE dla kolejnych epok.

Wywołujemy również funkcję `plot_predictions`, która rysuje wykresy pozwalające porównać wygląd danych treningowych i testowych z wartościami przewidzianymi przez sieć.

```
1 mlp = MLP([
2     {"input_size": 1, "output_size": 15, "activation": "sigmoid"},
3     {"input_size": 15, "output_size": 15, "activation": "sigmoid"},
4     {"input_size": 15, "output_size": 1, "activation": "linear"},
5 ], weight_init="uniform")
6
7 mlp.train(X_train, y_train, 200000, 0.006, batch_size=30)
8
9 mlp.plot_predictions(X_train, y_train, X_test, y_test)
```

Listing 1: Przykładowa inicjalizacja i trening sieci

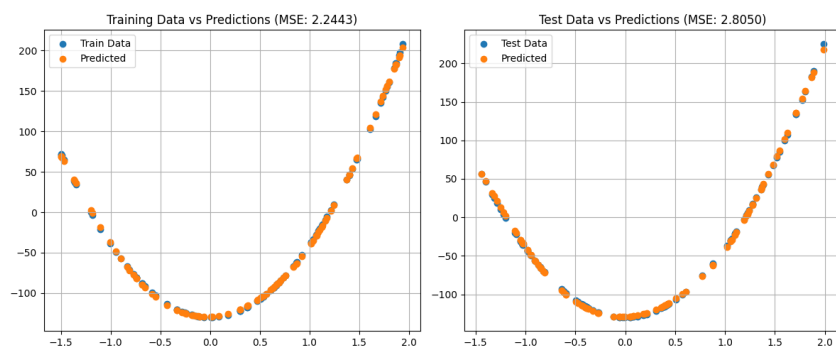
3.3 Wyniki i wnioski

Testy poprawności działania sieci przeprowadzono na trzech zbiorach danych: *square-simple*, *steps-small* oraz *multimodal-large*. Dla każdego przypadku mierzono błąd średniokwadratowy (MSE), który musiał osiągnąć wartości odpowiednio: 4, 4 oraz 40.

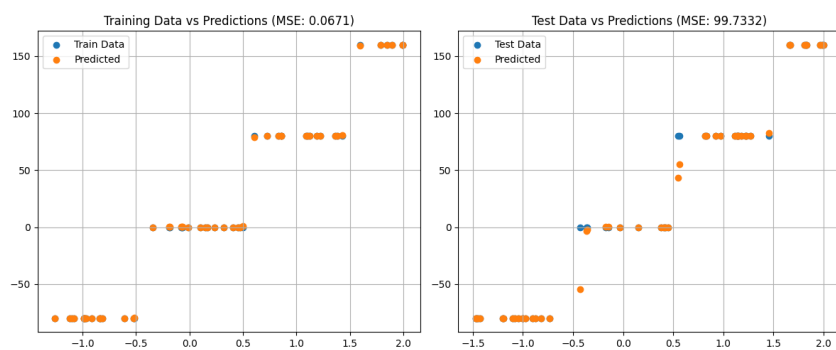
Dodatkowo, w przypadku problemów z nauką sieci, wykorzystano wizualizacje wag do analizy przyczyn niepowodzenia. Można było w ten sposób zauważyć na przykład momenty, gdy wagi bardzo szybko, nadmiernie rosły.

Dane *square-simple* były normalizowane, a pozostałe nie. Dla zbioru *square-simple* oraz *multimodal-large* została wykorzystana inicjalizacja wag `uniform`, a dla zbioru *steps-small* inicjalizacja wag `he`.

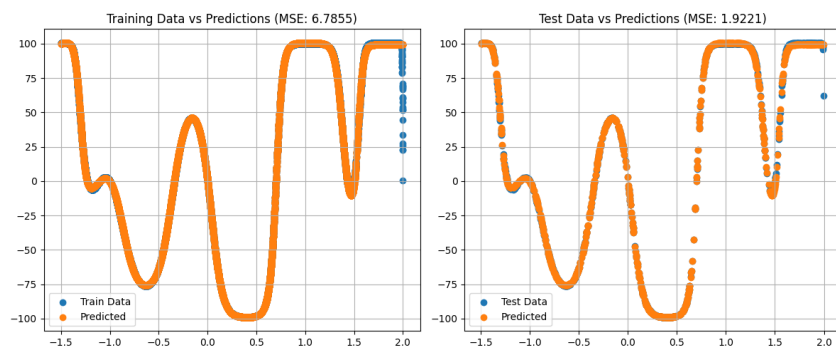
Uzyskanie dobrego wyniku na zbiorze testowym *steps-small* było praktycznie niemożliwe ze względu na przesunięcie zbioru testowego względem treningowego. Podobnie dla zbioru *multimodal-large* wartość MSE na zbiorze train jest nieco wyższa ze względu na nietypowy kształt z prawej strony.



Rysunek 7: Wyniki dla zbioru *square-simple*



Rysunek 8: Wyniki dla zbioru *steps-small*



Rysunek 9: Wyniki dla zbioru *multimodal-large*

zbiór danych	architektura	l. epok	learning rate	batch size	train MSE	test MSE
<i>square-simple</i>	1-15-15-1	200 000	0.006	30	2.244	2.805
<i>steps-small</i>	1-15-15-1	100 000	0.004	25	0.067	99.733
<i>multimodal-large</i>	1-35-35-1	10 000	0.005	120	6.786	1.922

Tabela 2: Uzyskane wartości MSE i wykorzystane parametry

4 NN3: implementacja momentu i normalizacji gradientu

4.1 Opis zadania

Celem zadania była implementacja dwóch technik usprawniających proces uczenia gradientowego sieci neuronowej: momentu oraz normalizacji gradientu metodą RMSProp. Obie metody zostały zaimplementowane zgodnie z teorią przedstawioną na wykładach oraz w materiałach dodatkowych.

Pierwsza z metod – moment – pozwala na przyspieszenie zbieżności poprzez uwzględnienie wcześniejszych kierunków aktualizacji wag, co zapobiega zatrzymywaniu się w płytkich minimach oraz niestabilnym oscylacjom.

Druga metoda – RMSProp – polega na normalizacji gradientu w oparciu o średnią kwadratów jego niedawnych wartości, co pozwala na dynamiczne dostosowywanie kroku uczenia do charakterystyki funkcji błędu.

4.2 Implementacja

Wybór optymalizacji został dodany jako parametr `mode` funkcji `train`, domyślnie jest `mode="standard"`, czyli bez optymalizacji. Do wyboru mamy: `standard`, `momentum`, `rms_prop`.

```
1 mlp_mom.train(X_train, y_train, 500000, 0.005, batch_size=100, mode="momentum")
2 mlp.train(X_train, y_train, 200000, 0.005, batch_size=100, mode="rmsprop")
```

Listing 2: Przykładowe wywołanie funkcji `train` z optymalizacją

4.3 Wyniki i wnioski

Testy poprawności działania sieci przeprowadzono na trzech zbiorach danych: *square-large*, *steps-large* oraz *multimodal-large*. Dla każdego przypadku mierzono błąd średniokwadratowy (MSE), który musiał osiągnąć wartości odpowiednio: 1, 3 oraz 9.

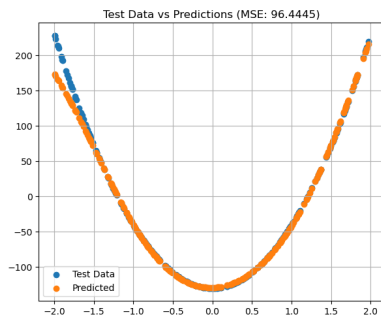
Dla każdego zbioru danych została przetestowana wersja bez optymalizacji, z optymalizacją `momentum` oraz z optymalizacją `rms_prop`. W wielu przypadkach wywoływałam funkcję `train` kilka razy z inną wartością `learning rate`, aby dotrenować model.

Największe wyzwanie stanowiło trenowanie sieci dla zbioru *square-large*, ponieważ dziedzina w zbiorze treningowym i testowym znacznie się różniła. W zbiorze treningowym dziedzina była z zakresu $[-1.5, 2]$, a w zbiorze testowym z zakresu $[-2, 2]$. Przez to wynik MSE na zbiorze testowym nie był zadowalający i uzyskanie małego MSE było praktycznie niemożliwe. Przykładowo w jednym z przypadków na zbiorze treningowym uzyskałam MSE równe 0.183, a na zbiorze testowym MSE było równe 61.881. Możemy zauważyć na wykresach, że poza fragmentem dziedziny $[-2, -1.5]$ wykres jest praktycznie perfekcyjnie dopasowany.

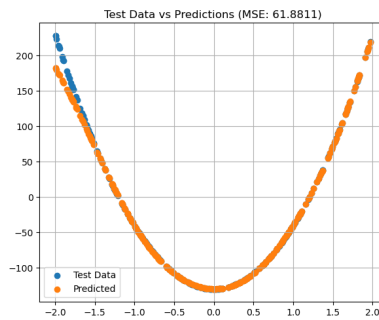
Dane *square-simple* były normalizowane, a pozostałe nie. Dla zbioru *multimodal-large* wykorzystałam funkcję `tanh` jako funkcję aktywacji. Dla zbioru *square-large* oraz *multimodal-large* została wykorzystana inicjalizacja wag `uniform`, a dla zbioru *steps-large* inicjalizacja wag `he`.

Porównując wszystkie wyniki, najlepiej działa optymalizacja `momentum`. Osiąga ona najlepsze wyniki. Interesującą obserwacją jest to, że wersja bez optymalizacji (`standard`) radzi sobie lepiej niż wersja z optymalizacją RMSProp.

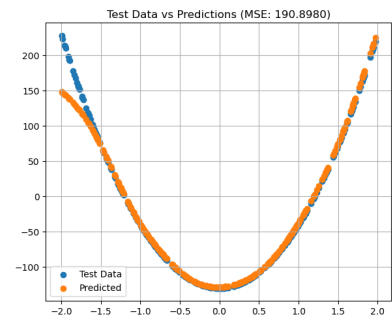
Wyniki dla zbioru *square-large*



Rysunek 10: Mode: standard



Rysunek 11: Mode: momentum

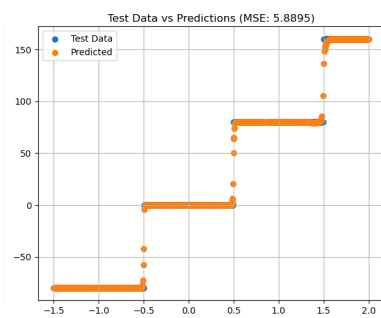


Rysunek 12: Mode: RMSprop

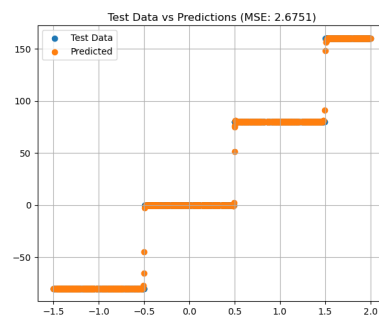
mode	architektura	l. epok	learning rate	batch size	train MSE	test MSE
standard	1-5-5-1	500 000	0.005	100	1.159	96.445
momentum	1-5-5-1	500 000	0.005	100	0.189	61.881
rmsprop	1-5-5-1	200 000	0.005	100	6.018	190.898

Tabela 3: Uzyskane wartości MSE i wykorzystane parametry dla zbioru *square-large*

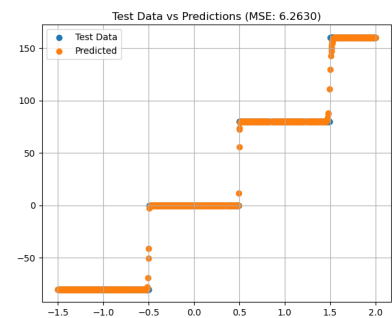
Wyniki dla zbioru *steps-large*



Rysunek 13: Mode: standard



Rysunek 14: Mode: momentum

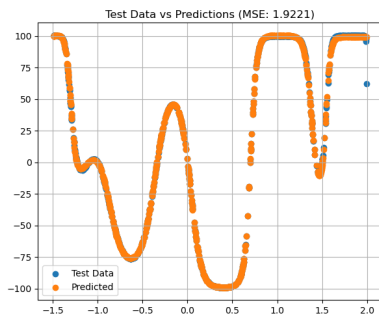


Rysunek 15: Mode: RMSprop

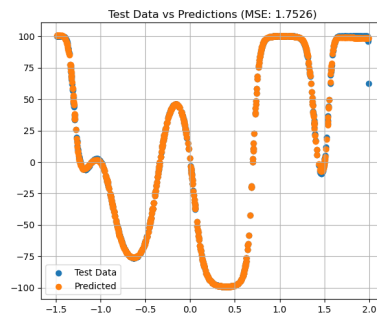
mode	architektura	l. epok	learning rate	batch size	train MSE	test MSE
standard	1-5-5-5-5-1	40 000	0.003	None	6.939	5.890
		60 000	0.001	None		
momentum	1-5-5-5-5-1	40 000	0.003	None	2.997	2.675
		60 000	0.001	None		
		20 000	0.001	None		
rmsprop	1-5-5-5-5-1	40 000	0.003	None	8.791	6.263
		60 000	0.001	None		

Tabela 4: Uzyskane wartości MSE i wykorzystane parametry dla zbioru *steps-large*

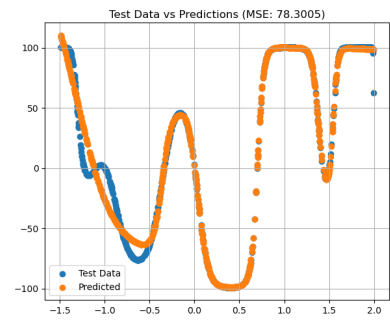
Wyniki dla zbioru *multimodal-large*



Rysunek 16: Mode: standard



Rysunek 17: Mode: momentum



Rysunek 18: Mode: RMSprop

mode	architektura	l. epok	learning rate	batch size	train MSE	test MSE
standard	1-35-1	10 000	0.005	120	6.786	1.922
momentum	1-35-1	10 000	0.005	120	6.575	1.753
rmsprop	1-35-1	10 000	0.005	120	85.551	78.301

Tabela 5: Uzyskane wartości MSE i wykorzystane parametry dla zbioru *multimodal-large*

5 NN4: rozwiązywanie zadania klasyfikacji

5.1 Opis zadania

Celem ćwiczenia było zaimplementowanie funkcji `softmax` dla warstwy wyjściowej sieci neuronowej oraz porównanie jej działania z klasyczną funkcją aktywacji - `sigmoid`. Dodatkowo, należało odpowiednio zmodyfikować algorytm uczenia, aby uwzględnić pochodną funkcji `softmax`. Umożliwia to rozwiązywanie problemów klasyfikacji.

5.2 Implementacja

To zadanie wymagało dodania kilku funkcjonalności do klasy MLP. Został dodany parametr `task` do klasy MLP. Może on przyjmować wartość `regression` lub `classification`, odpowiednio do rozwiązywania problemu regresji i klasyfikacji.

Funkcję `backpropagate` należało dostosować tak, aby TODOTODOTODOTODO

W funkcji `train` dane są dodatkowo one-hot-encodowane w przypadku problemów klasyfikacji. W tym celu stworzyłam funkcję `one_hot_encode`, która wykonuje ten proces.

Zmiany wymagał również sposób liczenia funkcji straty. W przypadku problemów klasyfikacji funkcją straty jest cross-entropia, a na koniec wyświetlamy f1-score. W tym celu zaimplementowana została funkcja `cross_entropy` oraz `f1_score`.

5.3 Wyniki i wnioski

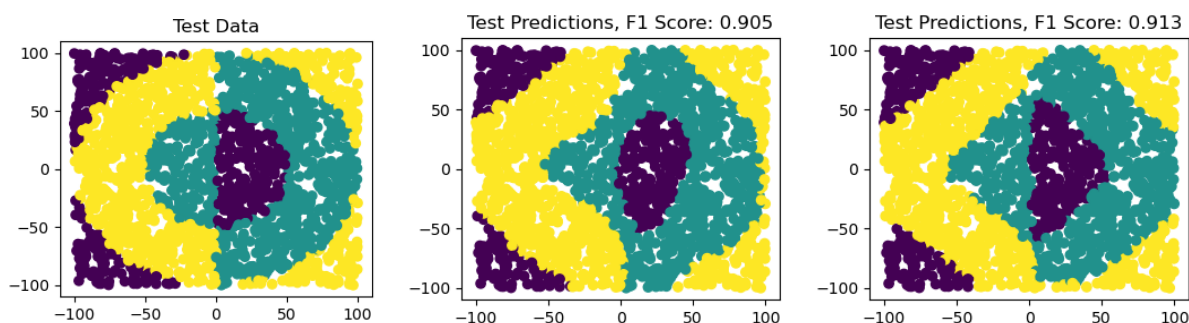
Eksperymenty zostały przeprowadzone na trzech zbiorach danych: *rings3-regular*, *easy*, *xor3-balance*. Są to zbiory dotyczące problemu klasyfikacji. Dla każdego przypadku mierzono f1-score, który musiał osiągnąć wartości odpowiednio: 0.75, 0.99 oraz 0.97. Jako funkcję straty przyjął `cross-entropy`.

Dla każdego zbioru została porównana implementacja z funkcją `softmax` i z funkcją `sigmoid` na warstwie wyjściowej.

Wszystkie dane były normalizowane. Dla zbioru *rings3-regular* oraz *easy* została wykorzystana inicjalizacja wag `uniform`, a dla zbioru *xor3-balance* inicjalizacja wag `xavier`. Dla zbiorów *rings3-regular* i *xor3-balance* trenowałam sieć z optymalizacją `momentum`, a dla zbioru *easy* bez optymalizacji.

Największe wyzwanie stanowiło znalezienie odpowiednich parametrów dla zbioru *xor3-balance*, a najłatwiej było znaleźć parametry dla zbioru *easy*.

Wyniki dla zbioru *rings3-regular*

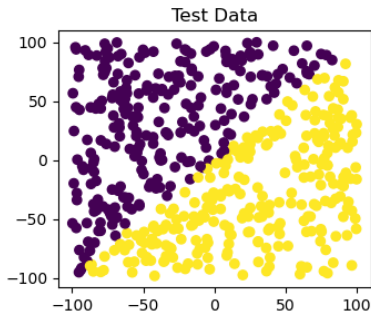


Rysunek 19: Zbiór treningowy

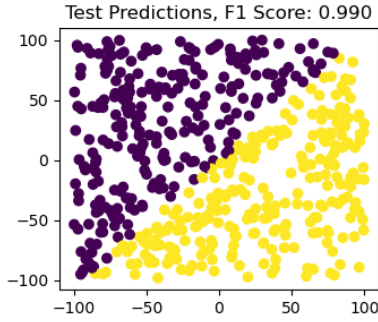
Rysunek 20: Funkcja softmax

Rysunek 21: Funkcja sigmoid

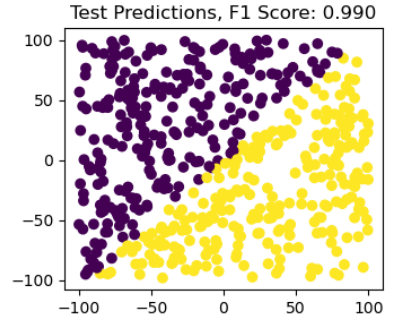
Wyniki dla zbioru *easy*



Rysunek 22: Zbiór treningowy

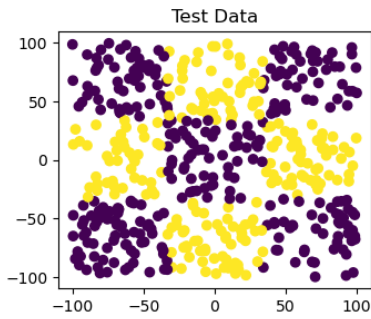


Rysunek 23: Funkcja softmax

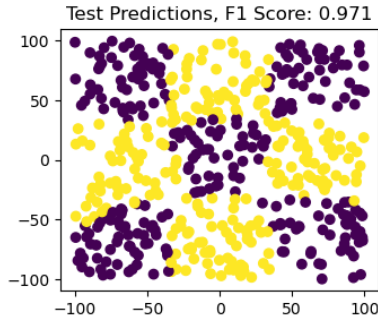


Rysunek 24: Funkcja sigmoid

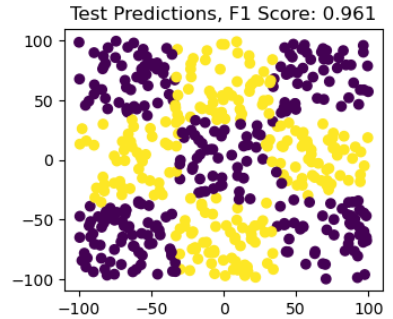
Wyniki dla zbioru *xor3-balance*



Rysunek 25: Zbiór treningowy



Rysunek 26: Funkcja softmax



Rysunek 27: Funkcja sigmoid

Podsumowanie wyników

zbiór danych	funkcja	architektura	liczba epok	learning rate	batch size	train f1-score	test f1-score
<i>rings3-regular</i>	softmax	2-10-10-3	1000	0.1	32	0.911	0.905
<i>rings3-regular</i>	sigmoid	2-10-10-3	1000	0.1	32	0.917	0.913
<i>easy</i>	softmax	2-5-5-2	5000	0.01	250	0.994	0.990
<i>easy</i>	sigmoid	2-5-5-2	5000	0.01	250	0.994	0.990
<i>xor3-balance</i>	softmax	2-9-9-2	5000	0.2	100	0.982	1.000
<i>xor3-balance</i>	sigmoid	2-9-9-2	15000	0.15	100	0.982	0.961

Tabela 6: Uzyskane wartości f1-score i wykorzystane parametry

6 NN5: testowanie różnych funkcji aktywacji

6.1 Opis zadania

Celem zadania było rozszerzenie istniejącej implementacji sieci neuronowej o obsługę różnych funkcji aktywacji: `sigmoid`, `linear`, `tanh` oraz `ReLU`. Konieczne było również dostosowanie procesu uczenia, tj. obliczania gradientów, do charakterystyki każdej z funkcji.

Dodatkowym aspektem było porównanie szybkości uczenia i skuteczności sieci w zależności od rodzaju funkcji aktywacji oraz liczby warstw ukrytych i neuronów. W tym celu testowano sieci z jedną, dwiema i trzema warstwami ukrytymi.

6.2 Implementacja

W tym zadaniu należało dodać kolejne funkcje aktywacji do pliku `ActivationFunctions.py` oraz ich pochodnych. Poniżej została przedstawiona dokładna implementacja poszczególnych funkcji.

```
1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
3
4 def sigmoid_derivative(z):
5     return z * (1 - z)
6
7 def linear(z):
8     return z
9
10 def linear_derivative(z):
11     return np.ones_like(z)
12
13 def tanh(z):
14     return np.tanh(z)
15
16 def tanh_derivative(z):
17     return 1 - z**2
18
19 def relu(z):
20     return np.maximum(0, z)
21
22 def relu_derivative(z):
23     return np.where(z > 0, 1, 0)
24
25 def softmax(x):
26     exp_x = np.exp(x - np.max(x))
27     return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

Rysunek 28: Implementacja funkcji aktywacji oraz ich pochodnych

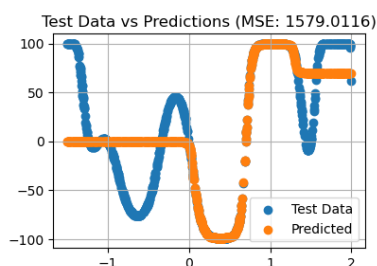
6.3 Wyniki i wnioski

Testy przeprowadzono dla zadania regresyjnego na zbiorze *multimodal-large*. Przetestowano wszystkie cztery funkcje aktywacji w trzech wariantach architektury sieci: 1-35-1, 1-35-35-1, 1-35-35-35-1. Aby móc porównać wyniki, trening był wykonany na tych samych parametrach przedstawionych poniżej.

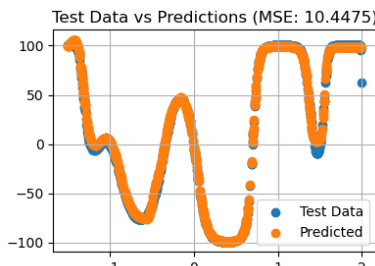
liczba epok	learning rate	batch size	normalizacja	mode	weight init
10 000	0.005	120	nie	standard	uniform

Tabela 7: Wykorzystane parametry

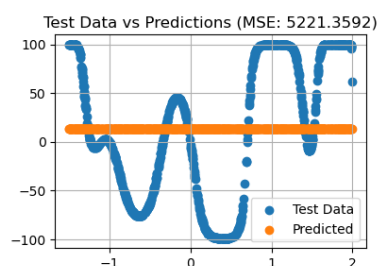
Testy dla funkcji sigmoid



Rysunek 29: Arch. 1-35-1

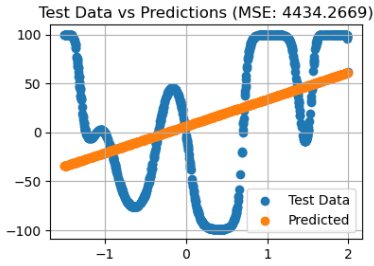


Rysunek 30: Arch. 1-35-35-1

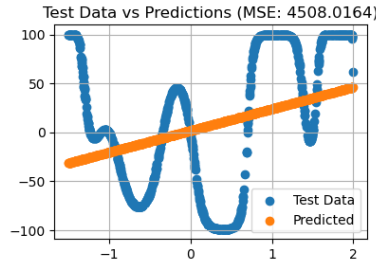


Rysunek 31: Arch. 1-35-35-35-1

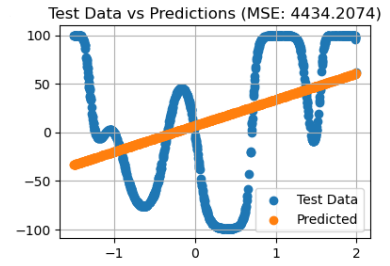
Testy dla funkcji linear



Rysunek 32: Arch. 1-35-1

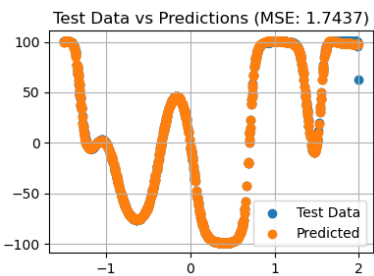


Rysunek 33: Arch. 1-35-35-1

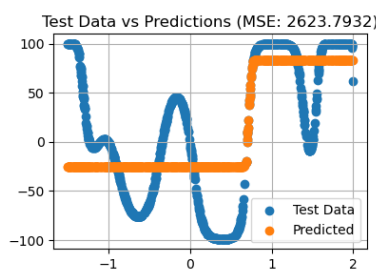


Rysunek 34: Arch. 1-35-35-35-1

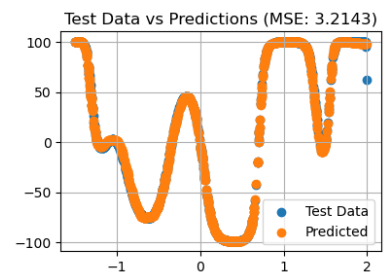
Testy dla funkcji tanh



Rysunek 35: Arch. 1-35-1

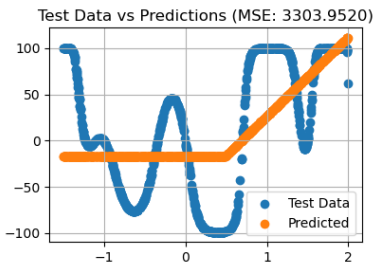


Rysunek 36: Arch. 1-35-35-1

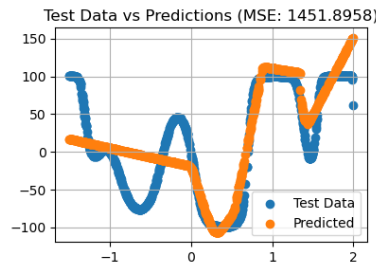


Rysunek 37: Arch. 1-35-35-35-1

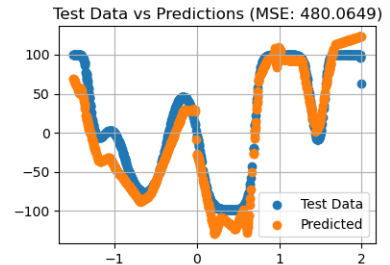
Testy dla funkcji ReLU



Rysunek 38: Arch. 1-35-1



Rysunek 39: Arch. 1-35-35-1



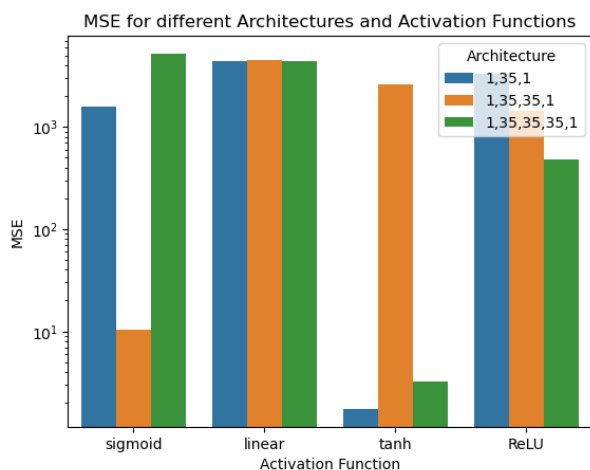
Rysunek 40: Arch. 1-35-35-35-1

Porównanie wszystkich wyników

Wszystkie wyniki zostały porównane na jednym wykresie. Wyniki bardzo różniły się między sobą, więc zastosowano skalę logarymiczną na osi y. Do dalszej analizy wybrano opcje:

- **opcja 1:** funkcja: `sigmoid`, architektura: 1-35-35-1
- **opcja 2:** funkcja: `tanh`, architektura: 1-35-1

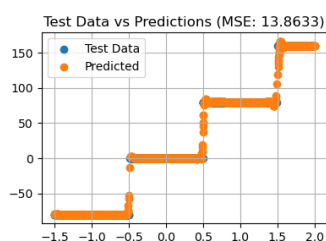
Dla tych dwóch przypadków zostały przeprowadzone testy dla zbiorów: *steps-large*, *rings5-regular* oraz *rings3-regular*.



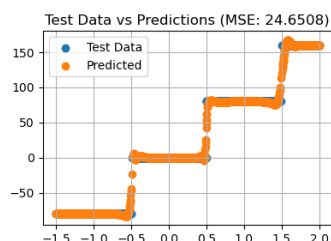
funkcja	architektura	test MSE
sigmoid	1,35,1	1579.01
sigmoid	1,35,35,1	10.45
sigmoid	1,35,35,35,1	5221.36
linear	1,35,1	4434.27
linear	1,35,35,1	4508.02
linear	1,35,35,35,1	4434.21
tanh	1,35,1	1.74
tanh	1,35,35,1	2623.79
tanh	1,35,35,35,1	3.21
ReLU	1,35,1	3303.95
ReLU	1,35,35,1	1451.90
ReLU	1,35,35,35,1	480.06

Rysunek 41: Porównanie wyników

Wyniki dla zbioru *steps-large*

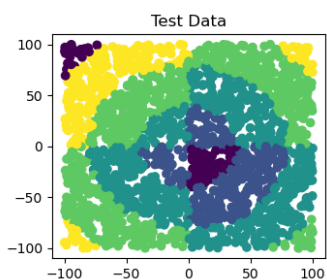


Rysunek 42: Opcja 1

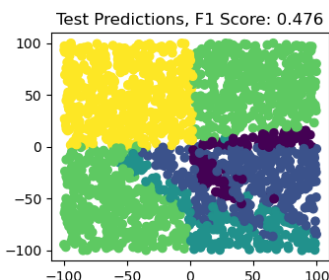


Rysunek 43: Opcja 2

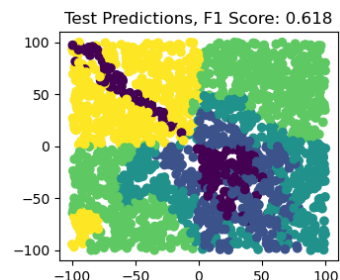
Wyniki dla zbioru *rings5-regular*



Rysunek 44: Zbiór train

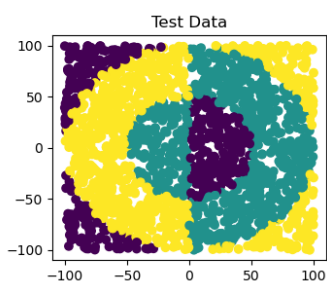


Rysunek 45: Opcja 1

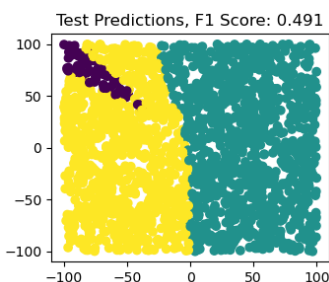


Rysunek 46: Opcja 2

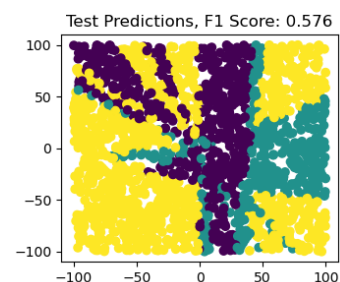
Wyniki dla zbioru *rings3-regular*



Rysunek 47: Zbiór train



Rysunek 48: Opcja 1



Rysunek 49: Opcja 2

7 NN6: zjawisko przeuczenia i regularyzacja

7.1 Opis zadania

Celem zadania było zaobserwowanie zjawiska przeuczenia (overfitting) w sieciach neuronowych oraz zaimplementowanie metod przeciwdziałających temu problemowi. Przeuczenie pojawia się, gdy model uczy się zbyt dobrze danych treningowych, tracąc zdolność generalizacji do danych testowych.

W tym celu rozbudowano implementację sieci o dwa mechanizmy regulujące proces uczenia:

- Regularyzacja wag – wprowadzająca karę do funkcji kosztu za duże wartości wag.
- Early stopping – zatrzymywanie procesu uczenia w momencie, gdy błąd na zbiorze walidacyjnym zaczyna rosnąć, co może sugerować początek przeuczenia.

Dzięki temu możliwe było porównanie skuteczności sieci przy użyciu różnych wariantów przeciwdziałania przeuczeniu oraz obserwacja, jak zmienia się jakość generalizacji modelu.

7.2 Implementacja

Dodanie tych dwóch funkcjonalności - regularyzacji wag i early stoppingu wymagało kilku zmian w kodzie klasy MLP.

W celu zaimplementowania regularyzacji wag, w metodzie `train` dodano parametr `regularization`, który umożliwia wybór typu regularyzacji (L1, L2 lub brak regularyzacji) oraz parametr `lambda_reg` określający siłę regularyzacji. W pętli aktualizującej wagi, przed wykonaniem kroku optymalizacji, gradient wag jest modyfikowany przez wywołanie nowej metody `apply_regularization`. Metoda ta przyjmuje aktualny gradient, wartości wag oraz wybrane parametry regularyzacji i zwraca zaktualizowany gradient z uwzględnieniem dodatkowego składnika regularyzacyjnego (dodanie $\lambda \cdot w$ dla L2 lub $\lambda \cdot \text{sign}(w)$ dla L1).

W celu wprowadzenia wczesnego zatrzymania treningu dodano parametr `early_stopping` do funkcji `train` oraz parametry `patience`, `X_test` i `y_test`, umożliwiające monitorowanie postępu uczenia na zbiorze testowym. W każdej epoce, gdy opcja early stopping jest włączona, liczymy błąd na zbiorze testowym. Jeśli spadnie on poniżej najlepszego dotychczasowego, resetujemy licznik epok bez poprawy i zapisujemy obecny model (wagi i biasy) do specjalnych buforów. W przeciwnym wypadku licznik jest zwiększany, a gdy przekroczy wartość zdefiniowaną przez `patience`, trening zostaje zatrzymany, a użytkownik informowany o najlepszym wyniku i odpowiadającej mu epoce. Dodatkowo, metoda `update_early_stopping` aktualizuje bufor najlepszych wyników.

7.3 Wyniki i wnioski

Eksperymenty przeprowadzono na czterech zbiorach danych: *multimodal-sparse* (regresja), *rings5-sparse* (klasyfikacja), *rings3-balance* (klasyfikacja), *xor3-balance* (klasyfikacja). Każdy ze zbiorów został przetestowany na sześć sposobów. Możemy przeprowadzać trening w Early stopping lub bez i z regularyzacją L1, L2 lub bez niej. Została przetestowana każda kombinacja tych możliwości. Poniżej zostały przedstawione wykorzystane parametry. Dla każdego zbioru parametry były takie same w każdym z przypadków, co umożliwiło dobre porównanie wyników. Dane nie były normalizowane.

zbiór danych	architektura	liczba epok	learning rate	batch size	mode	weight init	patience
multimodal-sparse	1-35-35-1	20 000	0.005	10	momentum	xavier	1000
rings5-sparse	2-35-35-5	20 000	0.005	10	momentum	xavier	1000
rings3-balance	2-35-35-3	20 000	0.005	500	momentum	xavier	1000
xor3-balance	2-35-35-2	20 000	0.005	500	momentum	xavier	1000

Tabela 8: Wykorzystane parametry

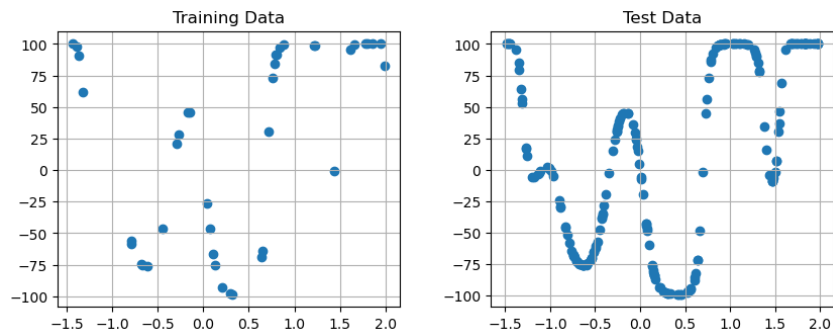
opcja	regularyzacja	early stopping
1	brak	tak
2	brak	nie
3	L1	tak
4	L1	nie
5	L2	tak
6	L2	nie

Tabela 9: Testowane kombinacje

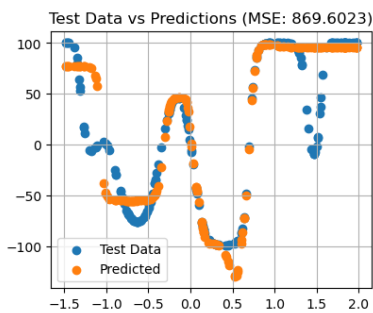
Dla zbioru *multimodal-sparse* najlepsze wyniki osiągnięto dla regularyzacji L1 bez early stoppingu. W różnych przypadkach regularyzacji, early stopping ma czasem pozytywny, a czasem negatywny wpływ.

W przypadku zbiorów klasyfikacyjnych, czyli *rings5-sparse*, *rings3-balance* i *xor3-balance* lepsze wyniki są osiągane bez early stoppingu. Regularyzacja L1 w tym przypadku daje najgorsze wyniki. Najlepsze wyniki są osiągane bez early stoppingu i bez regularyzacji. Regularyzacja L2 również przynosi bardzo dobre rezultaty.

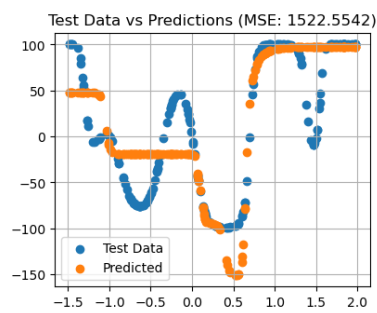
Testy dla zbioru *multimodal-sparse*



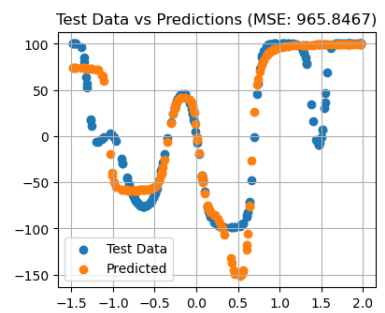
Rysunek 50: Zbiór danych



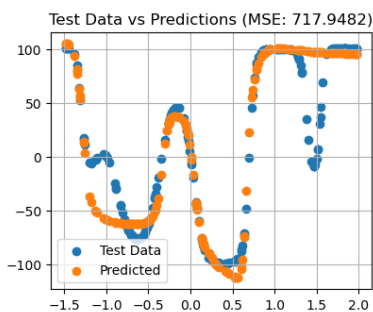
Rysunek 51: Opcja 1



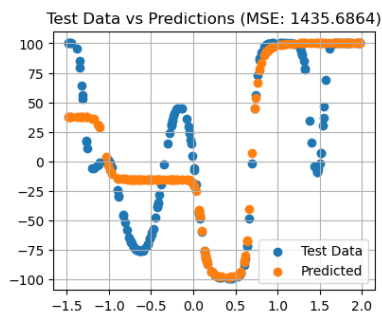
Rysunek 52: Opcja 2



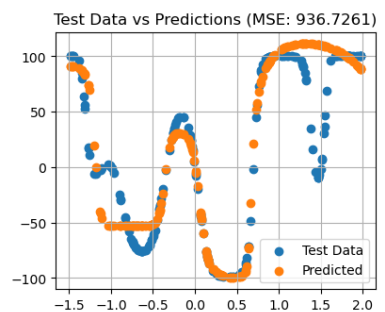
Rysunek 53: Opcja 3



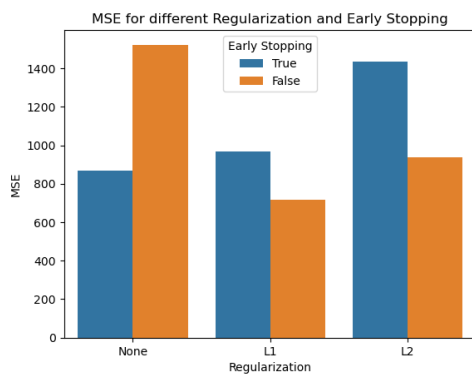
Rysunek 54: Opcja 4



Rysunek 55: Opcja 5



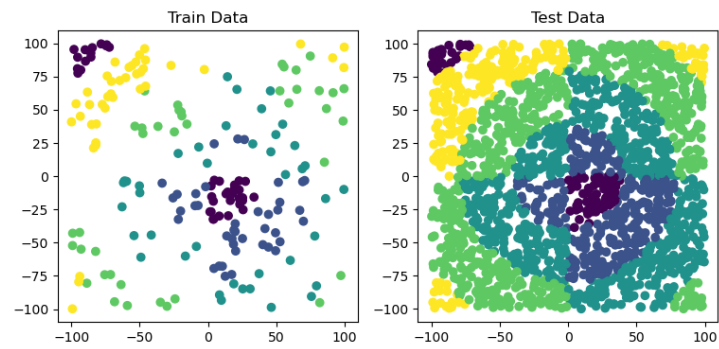
Rysunek 56: Opcja 6



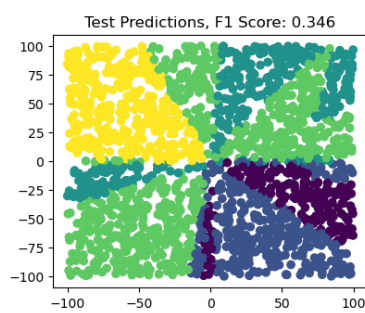
regularyzacja	early stopping	train MSE	test MSE
brak	tak	308.582	869.602
brak	nie	1094.272	1522.554
L1	tak	345.137	965.847
L1	nie	279.086	717.948
L2	tak	1200.646	1435.686
L2	nie	411.068	936.726

Rysunek 57: Porównanie wyników

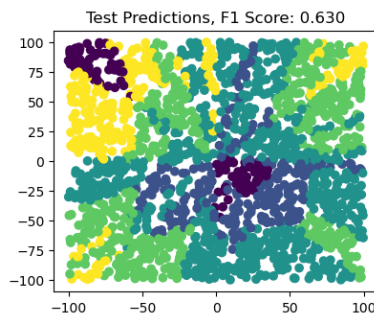
Testy dla zbioru *rings5-sparse*



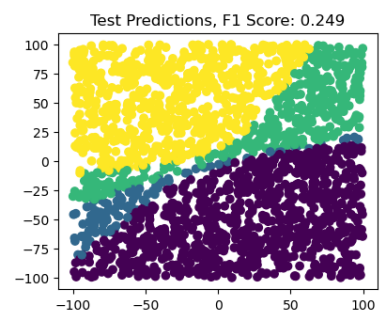
Rysunek 58: Zbiór danych



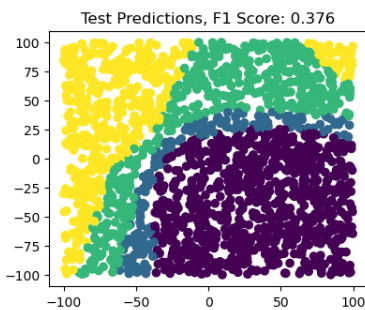
Rysunek 59: Opcja 1



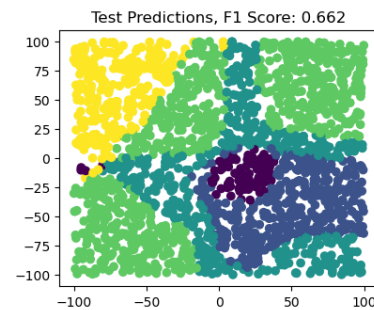
Rysunek 60: Opcja 2



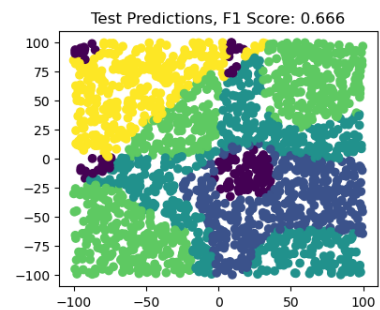
Rysunek 61: Opcja 3



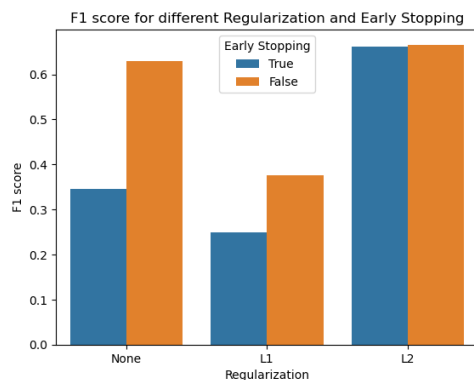
Rysunek 62: Opcja 4



Rysunek 63: Opcja 5



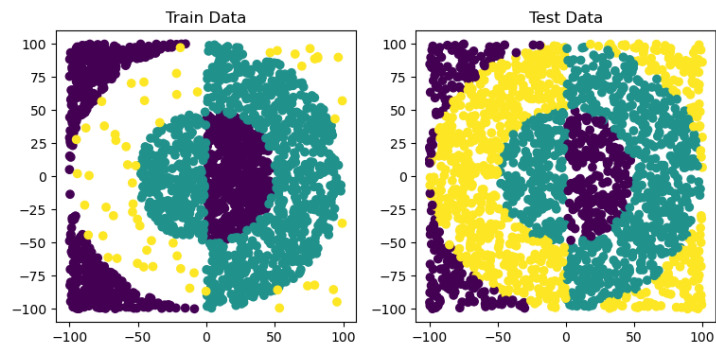
Rysunek 64: Opcja 6



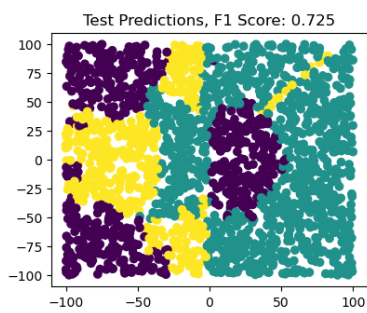
regularyzacja	early stopping	train F1	test F1
brak	tak	0.504	0.346
brak	nie	0.914	0.630
L1	tak	0.281	0.249
L1	nie	0.419	0.376
L2	tak	0.728	0.662
L2	nie	0.780	0.666

Rysunek 65: Porównanie wyników

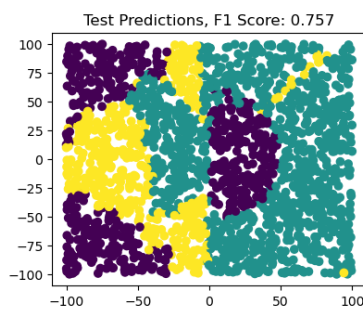
Testy dla zbioru *rings3-balance*



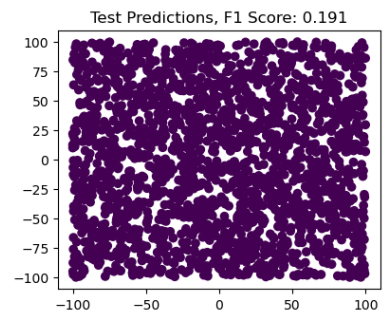
Rysunek 66: Zbiór danych



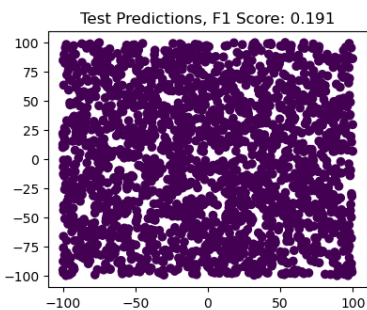
Rysunek 67: Opcja 1



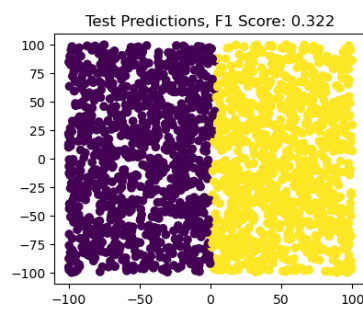
Rysunek 68: Opcja 2



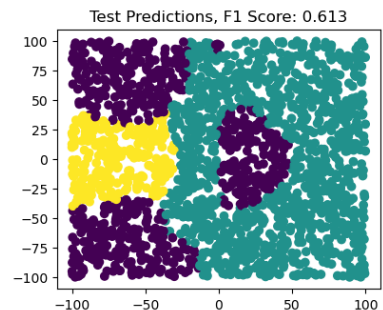
Rysunek 69: Opcja 3



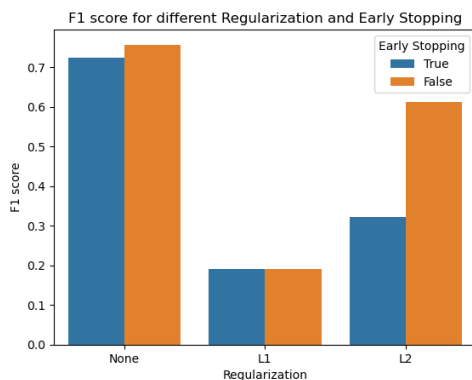
Rysunek 70: Opcja 4



Rysunek 71: Opcja 5



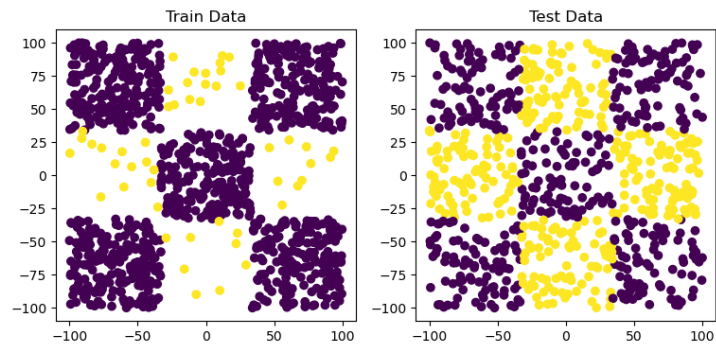
Rysunek 72: Opcja 6



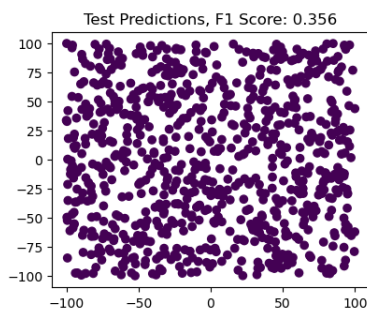
regularyzacja	early stopping	train F1	test F1
brak	tak	0.735	0.725
brak	nie	0.770	0.757
L1	tak	0.218	0.191
L1	nie	0.218	0.191
L2	tak	0.413	0.322
L2	nie	0.671	0.613

Rysunek 73: Porównanie wyników

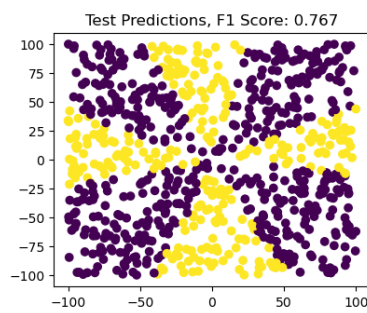
Testy dla zbioru *xor3-balance*



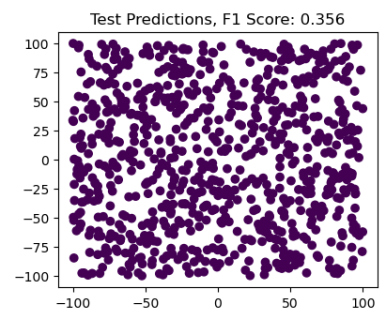
Rysunek 74: Zbiór danych



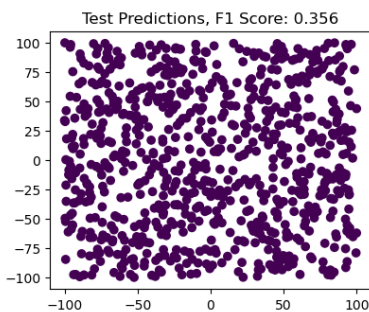
Rysunek 75: Opcja 1



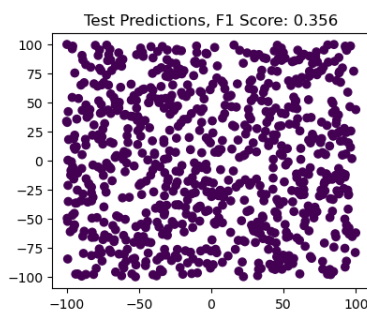
Rysunek 76: Opcja 2



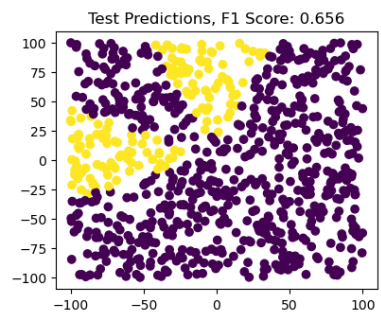
Rysunek 77: Opcja 3



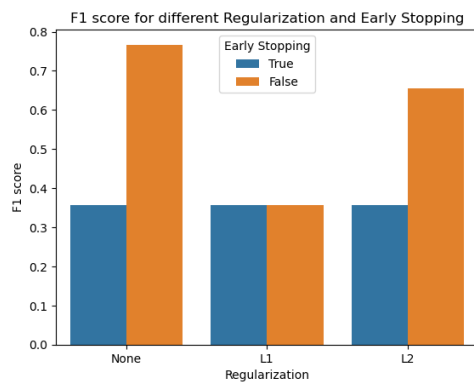
Rysunek 78: Opcja 4



Rysunek 79: Opcja 5



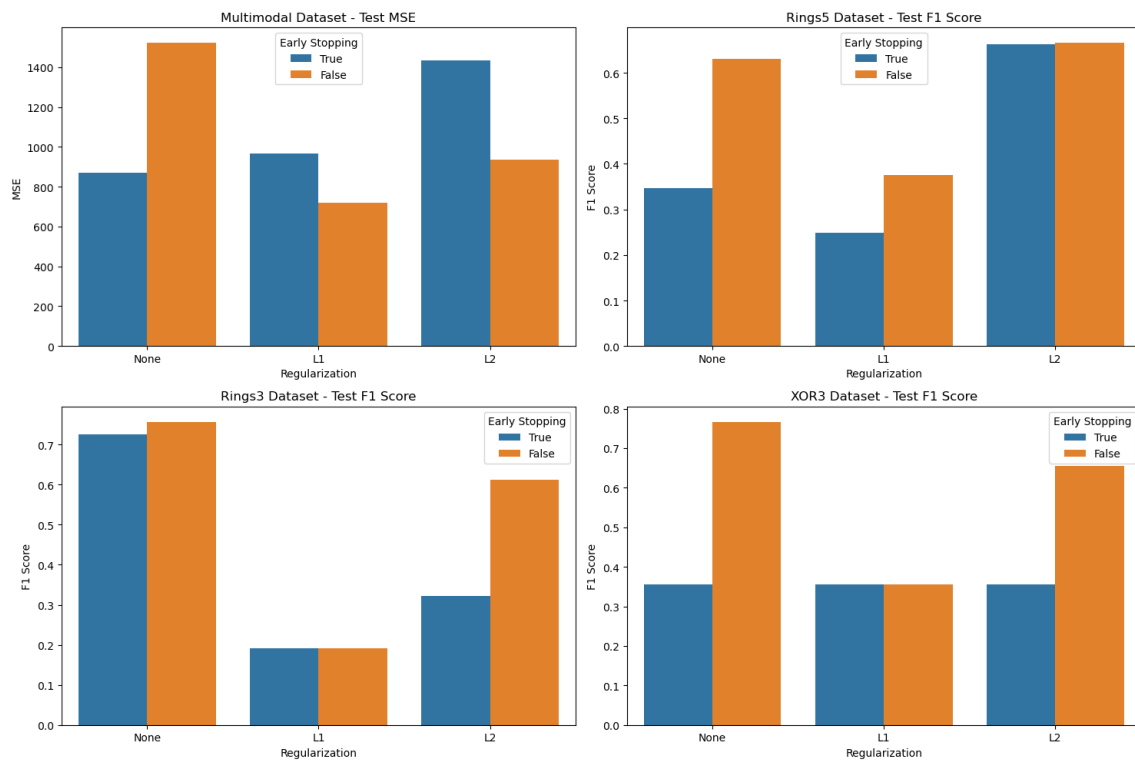
Rysunek 80: Opcja 6



regularyzacja	early stopping	train F1	test F1
brak	tak	0.488	0.356
brak	nie	0.693	0.767
L1	tak	0.488	0.356
L1	nie	0.488	0.356
L2	tak	0.488	0.356
L2	nie	0.750	0.656

Rysunek 81: Porównanie wyników

Porównanie wszystkich wyników



Rysunek 82: Zbiór danych